

Extending Classes

A major benefit of object-oriented programming is the ability to inherit classes and eliminate duplicate code. Inheritance allows you to define a new class based on an existing class.

Name:	Olivia	Name:	
	Bakke		

Content Learning Objectives

After completing this activity, students should be able to:

- Read and interpret UML class diagrams for an existing code base.
- Evaluate pros and cons for designs with multiple similar classes.
- Define inheritance and demonstrate how to extend a base class.

Process Skill Goals

During the activity, students should make progress toward:

- Working with all team members to reach consensus on hard questions. (Teamwork)



Model 1 UML Class Diagrams

Unified Modeling Language (UML) provides a standard way of visualizing how programs are designed (<http://www.uml.org/what-is-uml.htm>). For example, a *class diagram* is a graphical summary of the attributes and methods of a class.

```
class Cow:

    def __init__(self):
        self.position = (0, 0)
        self.respired = 0
        self.stomach = []

    def eat(self, item):
        if item == "grass":
            self.stomach.append(item)
            return "ate " + item
        else:
            return "This herbivore doesn't eat " + item

    def move(self, delta):
        self.position = (self.position[0] + delta[0],
                        self.position[1] + delta[1])
        return "This quadruped walked to " + str(self.position)

    def respire(self):
        self.respired += 1
        return "This cow respired through its nostrils."

    def speak(self):
        return "moo"
```

Cow
position : tuple respired : int stomach : list
<u>__init__()</u> eat(item : str) move(delta : tuple) respire() speak()

Questions (15 min)

Start time: 1:00

1. For each attribute and method name in the diagram find where it's defined in the code.

2. What are the attributes of Cow ? What are the methods of Cow ?

position, respired, and stomach methods are eat, move, respire, and speak

3. What is listed in each section of the UML class diagram?

a) Top section:

Name of the class

b) Middle section:

attributes

c) Bottom section:

methods

4. Consider the following class diagrams:

Cow	Horse	Lion	Pig
position : tuple respired : int stomach : list	position : tuple respired : int stomach : list	position : tuple respired : int stomach : list	position : tuple respired : int stomach : list
__init__() eat(item : str) move(delta : tuple) respire() speak()	__init__() eat(item : str) move(delta : tuple) nuzzle(position : tuple) respire() speak()	__init__() eat(item : str) move(delta : tuple) respire() speak()	__init__() eat(item : str) move(delta : tuple) respire() speak() wallow()

a) What attributes do the classes have in common?

position, respired, and stomach

b) What methods do the classes have in common?

eat, move, respire, and speak

c) What methods are unique to a particular class?

nuzzle and wallow

5. Quickly examine the source code for each of the classes to identify similarities and differences. Write one or two words in each table cell to summarize your findings.

	Cow	Horse	Lion	Pig
eat	grass	grass	meat	everything
move	walked	walks	walked	walked
nuzzle	N/A	close	N/A	N/A
respire	nostrils	muzzle	nostrils	snout
speak	moo	nay	ROOOAR	oink
wallow	N/A	N/A	N/A	in water

6. Consider what it would take to add a new method named `sleep` to each of the classes.

a) Describe the process of adding the same method to each source file.

have to add a `def sleep()` and could just return how it sleeps to each class

b) If a mistake is found later on, how would you correct the method?

would have to go through each one

c) What problems do you see with this approach as more classes are added?

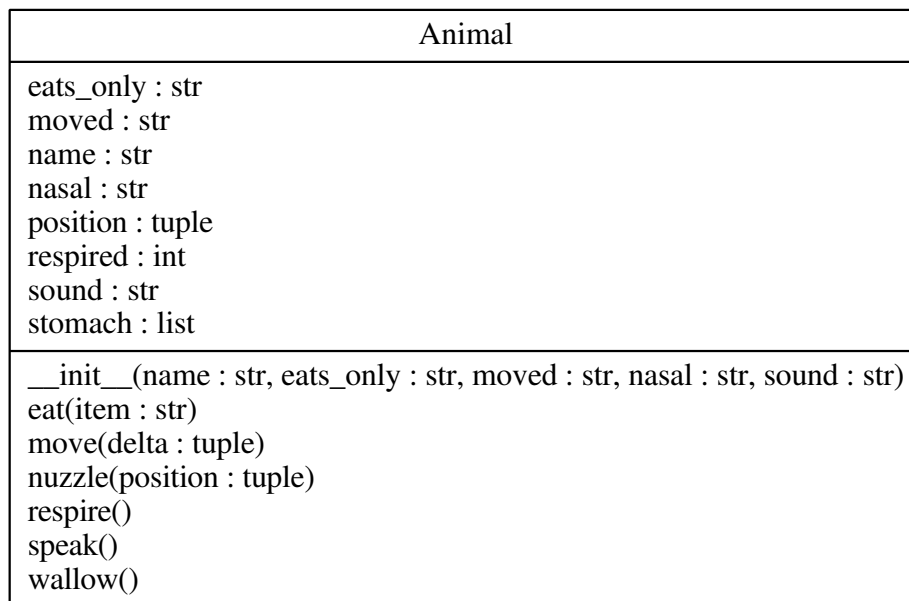
you have to do it a bunch of times

Model 2 Single-Class Approach

Given that the classes from Model 1 are similar, we could try combining them into a single class. In order to keep track of differences, we would need to store additional attributes. We could provide the information from Question #5 when creating an object:

```
angus = Animal("Cow", "grass", "walked", "nostrils", "moo")
```

The UML diagram below outlines this approach. As a team, discuss this design and become familiar with the accompanying source code.



Questions (10 min)

Start time: 1:10

7. Identify the three original attributes that were defined in Model 1.

position, respire, stomach

8. Identify the two methods that were NOT common to all four classes in Model 1.

nuzzle and wallow

9. Write a statement that creates an Animal object representing a lion. Assign it to a variable named simba

```
simba = Animal("Lion", "meat", "walked", "nostrils", "ROAR")
```

10. What methods does simba now have that it did not have in Model 1?

nuzzle and wallow

11. Describe 1–2 advantages the Model 2 design has compared to Model 1.

it's a lot faster and more versatile in how it can be used

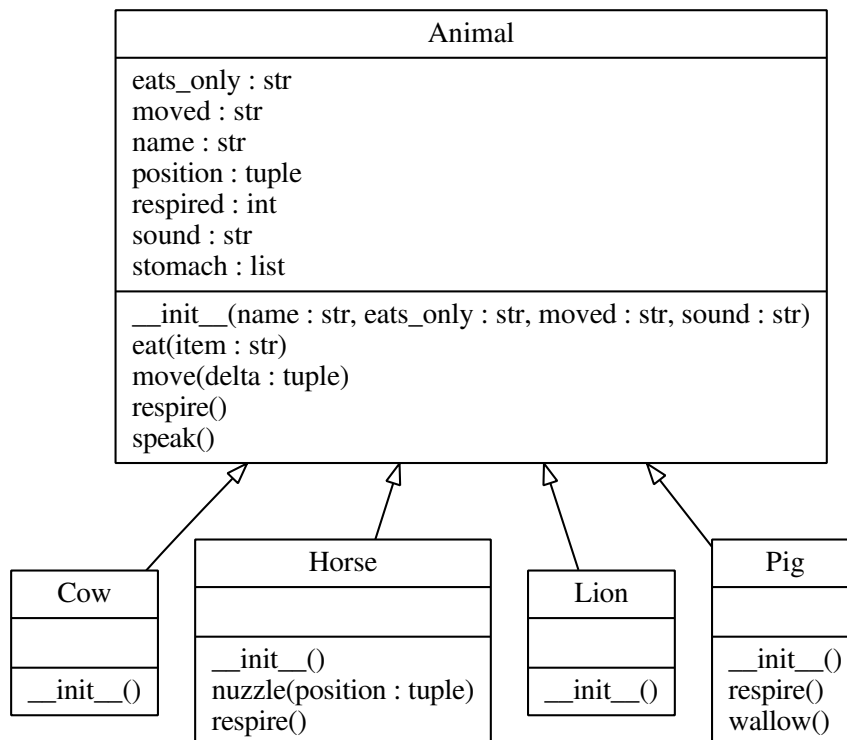
12. Describe 1–2 disadvantages the Model 2 design has compared to Model 1.

It's more difficult to make differences between the animals

Model 3 Derived Classes

We can improve the code from Model 2 by using *derived classes* for Cow, Horse, Lion, and Pig. These classes only contain the attributes and methods specific to them. Animal is a *base class* that contains attributes and methods they all have in common. This language feature is known as *inheritance*, because derived classes “inherit” attributes and methods from the base class.

The UML diagram below outlines this approach. As a team, discuss this design and become familiar with the accompanying source code.



Questions (20 min)

Start time: 1:15

13. Open the `lion.py` source file. How many methods are defined in the `Lion` class? List the name of each one.

None are defined in the `Lion` class

14. Type the following code into a Python Shell (in the same location as the source files). What methods are listed in the help?

```
from lion import Lion
help(Lion)
```

eat, move, respire, and speak

15. Write a statement that creates a `Lion` object. Assign it to a variable named `simba`. How is this statement different from Question #9?

`simba = Lion("meat", "walked", "nostrils", "ROAR")` - isn't made through the animal class

16. In a Python Shell, what is the value of `simba.speak()`? Where did this value come from?

`simba.speak() = ROAR` - comes from the method `speak` in the animal class

17. Does `simba` have any methods that it did not have in Model 1? Justify your answer.

No, because the `Lion` subclass did not add any

18. Based on the source files, how does the `__init__` method of `Animal` differ from the `__init__` methods of the derived classes?

in `Animal` it defined the attributes, and in the subclasses it constructed an instance of that class

19. What is the meaning of the built-in function `super()` that is used in the derived classes?

gives access to the methods of the `Animal` class in the subclasses

20. Describe 1–2 advantages the Model 3 design has compared to Model 2.

it allows for easy differentiation without having to redundantly recode everything every time